

Università
degli Studi di
Messina
DIPARTIMENTO DI INGEGNERIA

Master of **Engineering**
in Computer Science
A.Y. '25/'26

Industrial IoT

lab: OpenThread

prof. Giovanni Merlino

What is Thread?

Thread® is an **IPv6**-based networking protocol designed for **low-power** Internet of Things devices in an IEEE 802.15.4-2006 **wireless mesh** network, commonly called a Wireless Personal Area Network (*WPAN*). Thread is **independent** of other 802.15 mesh networking protocols, such a ZigBee, Z-Wave, and Bluetooth LE.

Core features include:

- **Simplicity** — simple ***installation***, start-up, and ***operation***
- **Security** — all devices ***authenticated*** and all communications ***encrypted***
- **Reliability** — ***self-healing*** mesh networking, with no single point of failure
    - and spread-spectrum techniques to provide immunity to interference
- **Efficiency** — low-power devices can sleep and operate on ***battery*** power for years
- **Scalability** — networks can scale up to ***hundreds*** of devices

OPENTHREAD

released by Google

- An **open-source** implementation of the Thread networking protocol
  - designed for low-power Internet of Things (IoT) devices

- Released by Google as a **reference** implementation of **Thread®** to
  - make the networking technology used in Google Nest products more *broadly* available to developers
  - *accelerate* the development of products for the ***connected home*** and commercial buildings

- With a narrow platform **abstraction layer** and a small memory footprint, OpenThread (OT) is highly **portable**
  - supporting both System-on-Chip (SoC) and Co-Processor (RCP, NCP) designs

OPENTHREAD

released by Google

- **Low-power** operation:
  - designed for devices that require low power consumption
    - making it ideal for battery-powered or **energy-harvesting** IoT devices
- **Scalability**:
  - supports large-scale networks (ability to handle thousands of nodes in a single network)
    - suitable for a wide range of IoT applications
      - (small) **home** networks
      - (large) industrial deployments
- **Security**:
  - offers **built-in** security features, protecting IoT devices from unauthorized access and malicious attacks
    - end-to-end encryption
    - secure **commissioning**
    - network-wide key rotation
- **Reliability**:
  - supports **mesh** networking
    - allowing devices to communicate with each other directly or through intermediate nodes
    - providing **multiple** paths for data transmission
      - increasing the reliability of the network
      - enabling self-healing in case of node failure

OPENTHREAD
released by Google

- **Application** layer **support**:
  - compatible with various application layer protocols, such as MQTT-SN, CoAP, and LWM2M
    - allowing for seamless integration with existing IoT applications and cloud services

- Platform **agnostic**:
  - can be implemented on various hardware platforms and operating systems
    - making it flexible for use with different devices

- Open-source and **community**-driven:
  - released under the permissive BSD-3-Clause license, which allows for open collaboration and innovation
    - anyone can contribute to the development and improvement of the protocol

Node Roles and Types

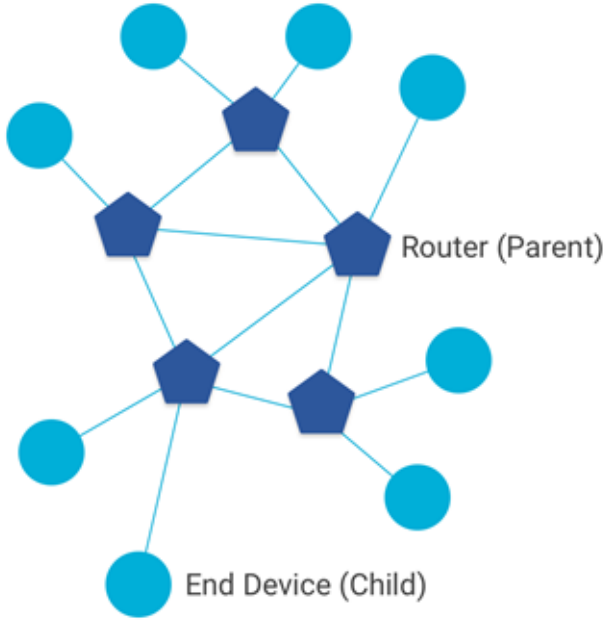
In a Thread network, nodes are split into two forwarding roles:

Router

- forwards packets for network devices
- provides secure commissioning services
  - for devices trying to join the network
- keeps its transceiver enabled at all times

End Device (ED)

- communicates primarily with a single Router
- does not forward packets for other network devices
- can disable its transceiver to reduce power



Device types

Full Thread Device (FTD)

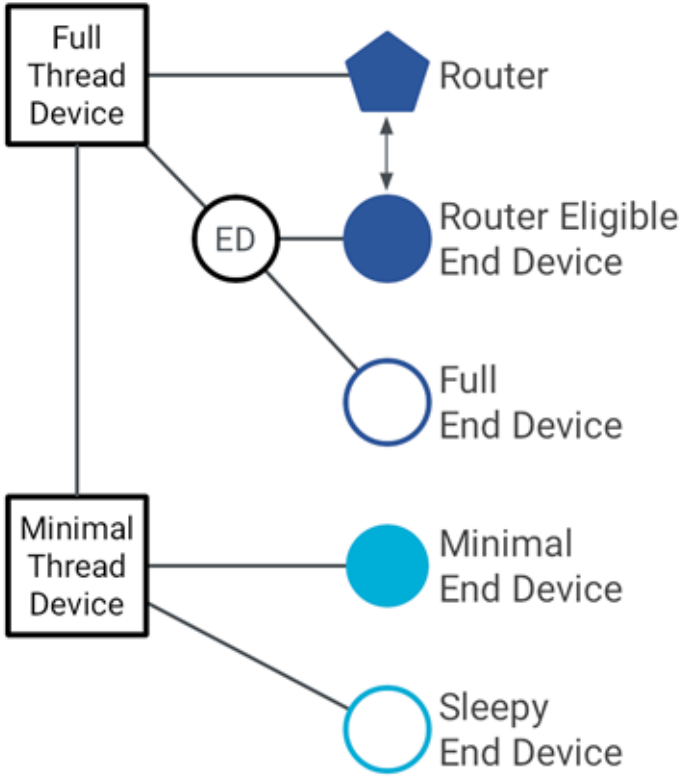
- always has its radio on, subscribes to the all-routers multicast address, and maintains IPv6 address mappings
- 3 types of FTDs:
  - Router
  - Router Eligible End Device (REED) — can be promoted to a Router
  - Full End Device (FED) — cannot be promoted to a Router

An FTD can operate as a Router (Parent) or an End Device (Child)

Minimal Thread Device

- does not subscribe to the all-routers multicast address and forwards all messages to its Parent
- 2 types of MTDs:
  - Minimal End Device (MED) — transceiver always on
    - does not need to poll for messages from its parent
  - Sleepy End Device (SED) — normally disabled
    - wakes on occasion to poll for messages from its parent

An MTD can **only** operate as an End Device (Child).

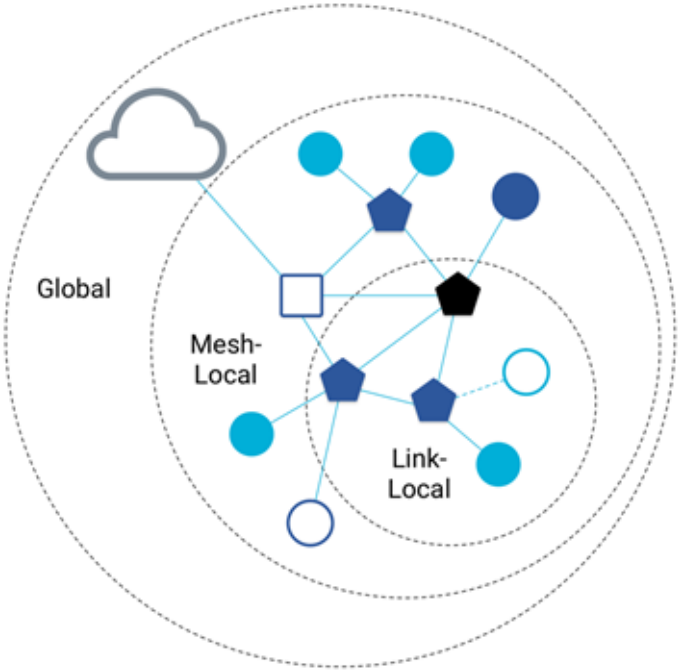


IPv6 Addressing

There are three scopes in a Thread network for unicast addressing:

- Link-Local — all interfaces reachable by a single radio transmission
- Mesh-Local — all interfaces reachable within the same Thread network
- Global — all interfaces reachable from outside a Thread network

The first two scopes correspond to prefixes designated by a Thread network. Link-Local have prefixes of `fe80::/16`, while Mesh-Local have prefixes of `fd00::/8`.



Network Discovery and Formation

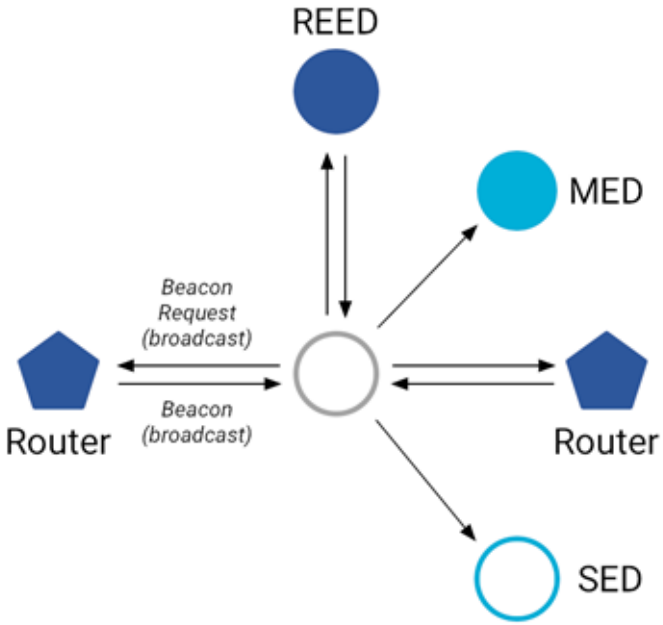
Thread networks are identified by three unique identifiers:

- 2-byte Personal Area Network ID (PAN ID)
- 8-byte Extended Personal Area Network ID (XPAN ID)
- A human-readable Network Name

When creating a new Thread network, or searching for an existing one to join, a Thread device performs an active scan for 802.15.4 networks within radio range:

1. The device broadcasts an 802.15.4 Beacon Request on a specific Channel.
2. In return, any Routers or Router Eligible End Devices (REEDs) in range broadcast a Beacon that contains their Thread network PAN ID, XPAN ID, and Network Name.
3. The device repeats the previous two steps for each Channel.

Once a Thread device has discovered all networks in range, it can either attach to an existing network, or create a new one if no networks are discovered.



Mesh Link Establishment

Thread uses the Mesh Link Establishment (MLE) protocol to configure links and disseminate information about the network to Thread devices.

In link configuration, MLE is used to:

- Discover links to neighboring devices
- Determine the quality of links to neighboring devices
- Establish links to neighboring devices
- Negotiate link parameters (device type, frame counters, timeout) with peers

MLE disseminates the following types of information to devices wishing to establish links:

- Leader data (Leader RLOC, Partition ID, Partition weight)
- Network data (on-mesh prefixes, address autoconfiguration, more-specific routes)
- Route propagation

Route propagation in Thread works similar to the Routing Information Protocol (RIP), a distance-vector routing protocol.

Create a new network / Join an existing network

- Create a new network

If the device elects to create a new network, it selects the least busy Channel and a PAN ID not in use by other networks, then becomes a Router and elects itself the Leader. This device sends MLE Advertisement messages to other 802.15.4 devices to inform them of its link state, and responds to Beacon Requests by other Thread devices performing an active scan.

- Join an existing network

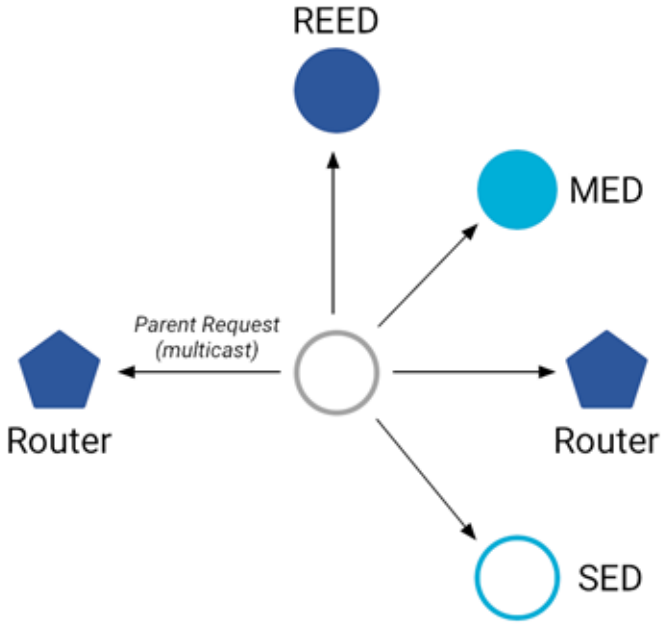
If the device elects to join an existing network, it configures its Channel, PAN ID, XPAN ID, and Network Name to match that of the target network via Thread Commissioning, then goes through the MLE Attach process to attach as a Child (End Device). This process is used for Child-Parent links.

1. The Child sends a multicast Parent Request to all neighboring Routers and REEDs in the target network.
2. All neighboring Routers and REEDs (if the Parent Request Scan Mask includes REEDs) send Parent Responses with information about themselves.
3. The Child chooses a Parent device and sends a Child ID Request to it.
4. The Parent sends a Child ID Response to confirm link establishment.

Parent Request

A Parent Request is a multicast request from the attaching device that is used to discover neighboring Routers and Router Eligible End Devices (REEDs) in the target network.

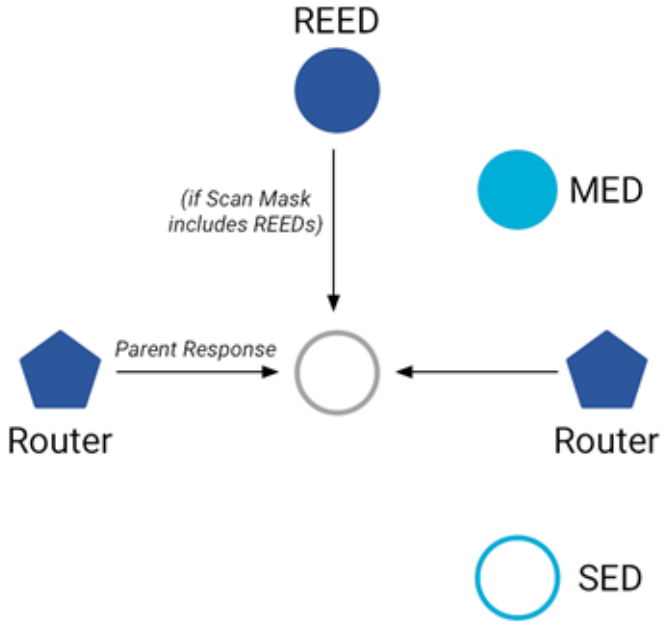
Parent Request Message Contents	
Mode	Describes the attaching device
Challenge	Tests the timeliness of the Parent Response to prevent replay attacks
Scan Mask	Limits the request to only Routers or to both Routers and REEDs



Parent Response

A Parent Response is a unicast response to a Parent Request that provides information about a Router or REED to the attaching device.

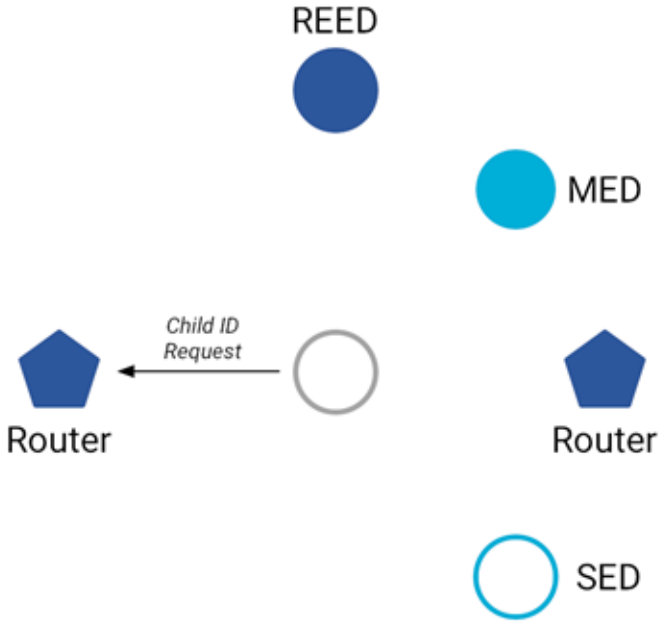
Parent Response Message Contents	
Version	Thread protocol version
Response	Copy of the Parent Request Challenge
Link Frame Counter	802.15.4 Frame Counter on the Router/REED
MLE Frame Counter	MLE Frame Counter on the Router/REED
Source Address	RLOC16 of the Router/REED
Link Margin	Receive signal quality of the Router/REED
Connectivity	Describes the Router/REED's level of connectivity
Leader Data	Information about the Router/REED's Leader
Challenge	Tests the timeliness of the Child ID Request to prevent replay attacks



Child ID Request

A Child ID Request is a unicast request from the attaching device (Child) that is sent to the Router or REED (Parent) for the purpose of establishing a Child-Parent link. If the request is sent to a REED, it upgrades itself to a Router before accepting the request.

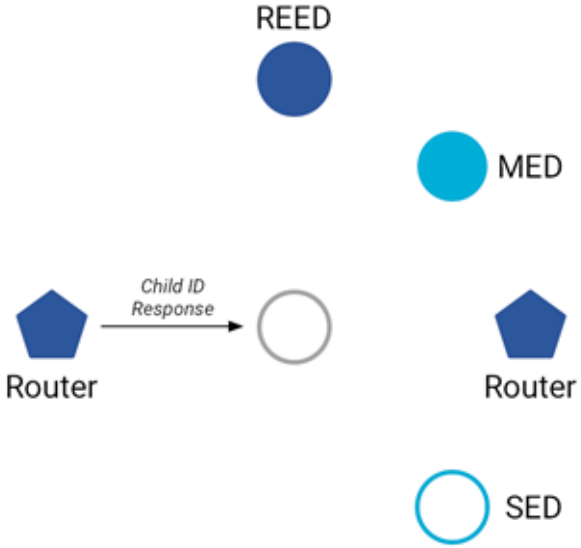
Child ID Request Message Contents	
Version	Thread protocol version
Response	Copy of the Parent Response Challenge
Link Frame Counter	802.15.4 Frame Counter on the Child
MLE Frame Counter	MLE Frame Counter on the Child
Mode	Describes the Child
Timeout	Inactivity duration before the Parent removes the Child
Address Registration (MEDs and SEDs only)	Register IPv6 addresses



Child ID Response

A Child ID Response is a unicast response from the Parent that is sent to the Child to confirm that a Child-Parent link has been established.

Child ID Response Message Contents	
Source Address	Parent's RLOC16
Address16	Child's RLOC16
Leader Data	Information about the Parent's Leader (RLOC, Partition ID, Partition weight)
Network Data	Information about the Thread network (on-mesh prefixes, address autoconfiguration, more-specific routes)
Route (REED only)	Route propagation
Timeout	Inactivity duration before the Parent removes the Child
Address Registration (MEDs and SEDs only)	Confirm registered addresses



Router Selection

Routers must form a Connected Dominating Set (CDS), which means:

1. There is a Router-only path between any two Routers.
2. Any one Router in a Thread network can reach any other Router by staying entirely within the set of Routers.
3. Every End Device in a Thread network is directly connected to a Router.

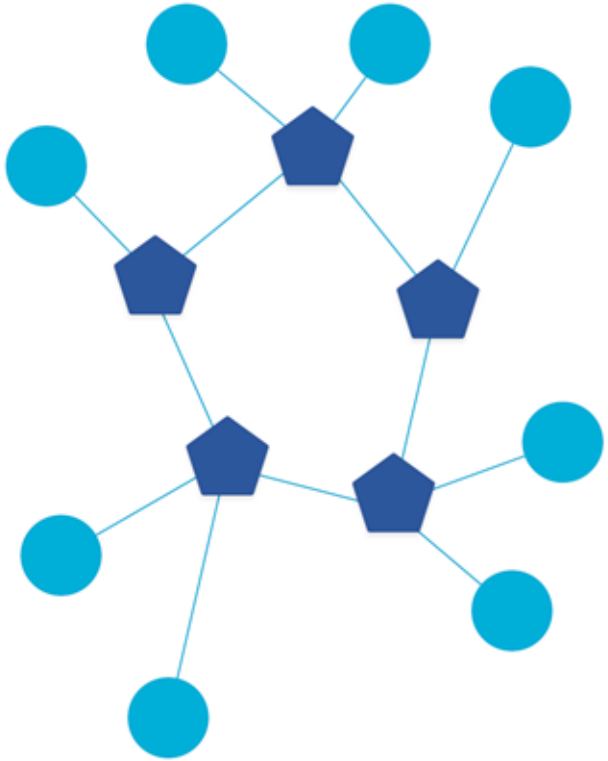
A distributed algorithm maintains the CDS, which ensures a minimum level of redundancy. Every device initially attaches to the network as an End Device (Child). As the state of the Thread network changes, the algorithm adds or removes Routers to maintain the CDS.

Thread adds Routers to:

- Increase coverage if the network is below the Router threshold of 16
- Increase path diversity
- Maintain a minimum level of redundancy
- Extend connectivity and support more Children

Thread removes Routers to:

- Reduce the Routing state below the maximum of 32 Routers
- Allow new Routers in other parts of the network when needed



Upgrade to a Router

After attaching to a Thread network, the Child device may elect to become a Router. Before initiating the MLE Link Request process, the Child sends an Address Solicit message to the Leader, asking for a Router ID. If the Leader accepts, it responds with a Router ID and the Child upgrades itself to a Router.

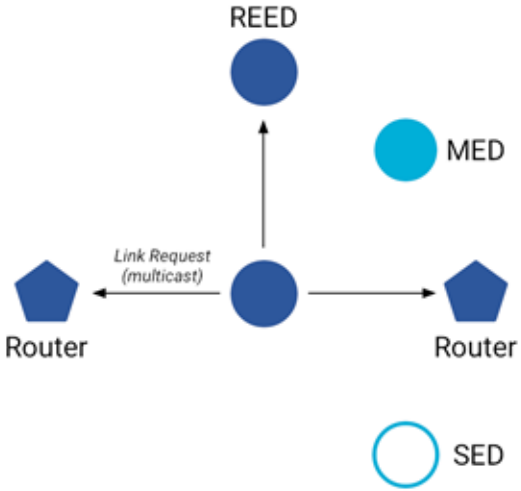
The MLE Link Request process is then used to establish bi-directional Router-Router links with neighboring Routers.

1. The new Router sends a multicast Link Request to neighboring Routers.
2. Routers respond with Link Accept and Request messages.
3. The new Router responds to each Router with a unicast Link Accept to establish the Router-Router link.

Link Request

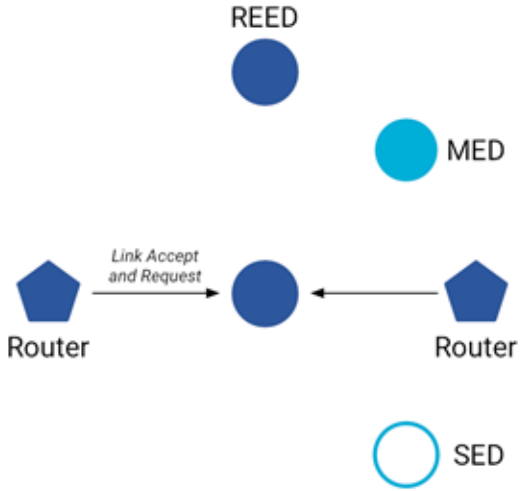
A Link Request is a request from the Router to all other Routers in the Thread network. When first becoming a Router, the device sends a multicast Link Request to `ff02::2`. Later, after discovering the other Routers via MLE Advertisements, the devices send unicast Link Requests.

Link Request Message Contents	
Version	Thread protocol version
Challenge	Tests the timeliness of the Link Response to prevent replay attacks
Source Address	RLOC16 of the sender
Leader Data	Information about the Router's Leader, as stored on the sender (RLOC, Partition ID, Partition weight)



Link Accept and Request

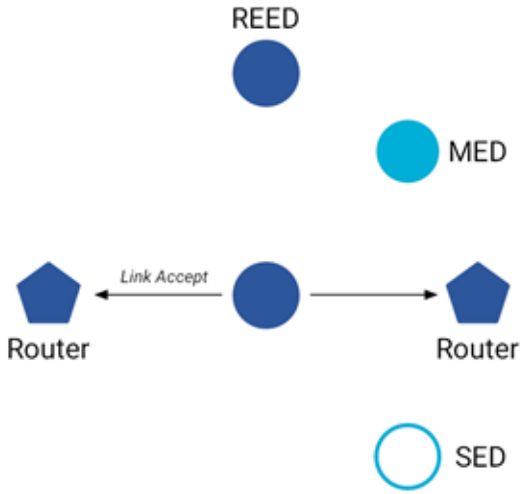
A Link Accept and Request is a combination of the Link Accept and Link Request messages. Thread uses this optimization in the MLE Link Request process to reduce the number of messages from four to three.



Link Accept

A Link Accept is a unicast response to a Link Request from a neighboring Router that provides information about itself and accepts the link to the neighboring Router.

Link Accept Message Contents	
Version	Thread protocol version
Response	Tests the timeliness of the Link Response to prevent replay attacks
Link Frame Counter	802.15.4 Frame Counter on the sender
MLE Frame Counter	MLE Frame Counter on the sender
Source Address	RLOC16 of the sender
Leader Data	Information about the Router's Leader, as stored on the sender (RLOC, Partition ID, Partition weight)



Downgrade to a REED

When a Router downgrades to a REED, its Router-Router links are disconnected, and the device initiates the MLE Attach process to establish a Child-Parent link.

See Join an existing network for more information on the MLE Attach process.

One-way receive links

In some scenarios, it may be necessary to establish a one-way receive link.

After a Router reset, neighboring Routers may still have a valid receive link with the reset Router. In this case, the reset Router sends a Link Request message to re-establish the Router-Router link.

An End Device may also wish to establish a receive link with neighboring non-Parent Routers to improve multicast reliability. We'll learn more about this when we get to Multicast Routing.

Set up a demo with Docker

Pull the Docker image

Once Docker is installed, open a terminal window and pull the `openthread/environment` Docker image. This image features OpenThread and OpenThread Daemon pre-built and ready to use for this Codelab.

```
docker pull openthread/environment:latest
```

Note that it may take a few minutes to completely download.

In a terminal window, start a Docker container from the image and connect to its `bash` shell:

```
docker run --name codelab_otsim_ctnr -it --sysctl net.ipv6.conf.all.disable_ipv6=0    --cap-add=net_admin \
openthread/environment bash
```

Note the flags, which are required for this Codelab:

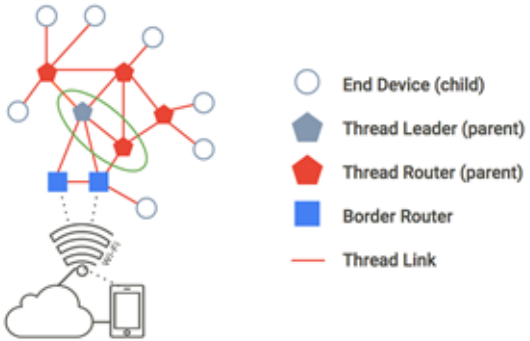
- `--sysctl net.ipv6.conf.all.disable_ipv6=0` — this enables IPv6 within the container
- `--cap-add=net_admin` — enables the NET_ADMIN capability, which allows you to execute network-related operations, such as adding IP routes

Simulate a Thread network

Start Node 1

If you haven't done so already, in a terminal window, start the Docker container and connect to its `bash` shell:

```
docker run --name codelab_otsim_ctnr -it --rm \
  --sysctl net.ipv6.conf.all.disable_ipv6=0 \
  --cap-add=net_admin openthread/environment bash
```



In the Docker container, spawn the CLI process for an emulated Thread devic the `ot-cli-ftd` binary.

```
root@c0f3912a74ff:/# /openthread/build/examples/apps/cli/ot-cli-ftd 1
```

Simulate a Thread network: Start Node 1

`dataset init new` creates a new Operational Dataset with random values. Your values will differ from the values shown in this Codelab. Replace the values in the Codelab example commands with the values from your Operational Dataset.

```
> dataset init new
        Done
> dataset
        Active Timestamp: 1
        Channel: 20
        Channel Mask: 07fff800
        Ext PAN ID: d6263b6d857647da
        Mesh Local Prefix: fd61:2344:9a52:ede0/64
        Network Key: e4344ca17d1dca2a33f064992f31f786
        Network Name: OpenThread-c169
        PAN ID: 0xc169
        PSKc: ebb4f2f8a68026fc55bcf3d7be3e6fe4
        Security Policy: 0, onrcb
        Done
```

Simulate a Thread network: Start Node 1

Commit this dataset as the active one:

```
> dataset commit active
    Done
```

Bring up the IPv6 interface:

```
> ifconfig up
    Done
```

Start Thread protocol operation:

```
> thread start
    Done
```

Wait a few seconds and verify that the device has become the Thread Leader. The Leader is the device responsible for managing router ID assignment.

```
> state
    leader
    Done
```

View the IPv6 addresses assigned to Node 1's Thread interface (your output will be different):

```
> ipaddr
    fd61:2344:9a52:ede0:0:ff:fe00:fc00
    fd61:2344:9a52:ede0:0:ff:fe00:5000
    fd61:2344:9a52:ede0:d041:c5ba:a7bc:5ce6
    fe80:0:0:0:94da:92ea:1353:4f3b
    Done
```

Note the specific IPv6 address types:

Begins with fd = mesh-local

Begins with fe80 = link-local

Mesh-local address types are classified further:

Contains ff:fe00 = Router Locator (RLOC)

Does not contain ff:fe00 = Endpoint Identifier (EID)

Identify the EID in your console output make a note of it for later use. In the sample output above, the EID is:

fd61:2344:9a52:ede0:d041:c5ba:a7bc:5ce6

Simulate a Thread network: Start Node 1

These are IPv6 addresses:

1. `fdac:8798:d14c:a47a:0:ff:fe00:fc00` and `fdac:8798:d14c:a47a:0:ff:fe00:cc00`: These are **private IPv6 addresses** (ULA), used within local networks.

2. `fdac:8798:d14c:a47a:9458:bd15:9d3e:e57`: Another **private IPv6 address** (ULA), identifying a different device on the network.

3. `fe80:0:0:0:286e:6951:9397:8359`: This is a **link-local IPv6 address**, used for communication within a single local network.

Simulate a Thread network: Start Node 1

```
root@56872738332e:/# /openthread/build/examples/apps/cli/ot-cli-ftd 1
> dataset init new
Done
> dataset
Active Timestamp: 1
Channel: 11
Channel Mask: 0x07fff800
Ext PAN ID: a20a6b93c5cb431d
Mesh Local Prefix: fd21:8892:8f38:9713::/64
Network Key: 9754ff0772f6fbd7403d4100076a55a8
Network Name: OpenThread-9ef0
PAN ID: 0x9ef0
PSKc: 29829770fecbbaef7d62f70671d3514c
Security Policy: 672 onrc
Done
> dataset commit active
Done
> ifconfig up
Done
> thread start
Done
> state
leader
Done
>
```

Simulate a Thread network: Start Node 2

Start Node 2

Open a new terminal and execute a bash shell in the currently running Docker container to use for Node 2.

```
docker exec -it codelab_otsim_ctnr bash
```

At this new bash prompt, spawn the CLI process with the argument 2. This is your second emulated Thread device:

```
root@c0f3912a74ff:/# /openthread/build/examples/apps/cli/ot-cli-ftd 2
```

Note: If you don't see the > prompt after running this command, press enter.

Configure the Thread Network Key and PAN ID, using the same values as Node 1's Operational Dataset:

```
> dataset networkkey e4344ca17d1dca2a33f064992f31f786
    Done
> dataset panid 0xc169
    Done
```

Commit this dataset as the active one:

```
> dataset commit active
    Done
```

Simulate a Thread network: Start Node 2

Bring up the IPv6 interface:

```
> ifconfig up
    Done
```

Start Thread protocol operation:

```
> thread start
    Done
```

The device will initialize itself as a Child. A Thread Child is equivalent to an End Device, which is a Thread device that transmits and receives unicast traffic only with a Parent device.

```
> state
    child
    Done
```

Within 2 minutes you should see the state switch from child to router. A Thread Router is capable of routing traffic between Thread devices. It is also referred to as a Parent.

```
> state
    router
    Done
```

Simulate a Thread network: Start Node 2

```
root@56872738332e:/# /openthread/build/examples/apps/cli/ot-cli-ftd 2
> dataset networkkey 9754ff0772f6fbd7403d4100076a55a8
Done
> dataset panid 0x9ef0
Error 35: InvalidCommand
> dataset panid 0x9ef0
Done
> dataset commit active
Done
> ifconfig up
Done
> thread start
Done
> state
detached
Done
> state
detached
Done
> state
detached
Done
> state
child
Done
```

```
> state
child
Done
> state
router
Done
>
```

Simulate a Thread network: Verify the network

Verify the network

An easy way to verify the mesh network is to look at the router table.

1. Check connectivity

On Node 2, get the RLOC16. The RLOC16 is the last 16 bits of the device's RLOC IPv6 address.

```
> rloc16
        5800
        Done
```

On Node 1, check the router table for Node 2's RLOC16. Make sure Node 2 has switched to the router state first.

```
> router table
        | ID | RLOC16 | Next Hop | Path Cost | LQ In  | LQ Out  | Age | Extended MAC     |
        +----+--------+----------+-----------+--------+---------+-----+------------------+
        | 20 | 0x5000 |       63 |         0 |      0 |       0 |   0 | 96da92ea13534f3b |
        | 22 | 0x5800 |       63 |         0 |      3 |       3 |  23 | 5a4eb647eb6bc66c |
```

Simulate a Thread network: Verify the network

Node 2's RLOC of `0x5800` is found in the table, confirming that it is connected to the mesh.

2. Ping Node 1 from Node 2

Verify connectivity between the two emulated Thread devices. In Node 2, `ping` the EID assigned to Node 1:

```
> ping fd61:2344:9a52:ede0:d041:c5ba:a7bc:5ce6
 > 16 bytes from fd61:2344:9a52:ede0:d041:c5ba:a7bc:5ce6: icmp_seq=1 hlim=64 time=12ms
```

Simulate a Thread network: Verify the network

```
> router table
| ID | RLOC16 | Next Hop | Path Cost | LQ In | LQ Out | Age | Extended MAC     | Link |
+----+--------+----------+-----------+-------+--------+-----+------------------+------+
| 10 | 0x2800 |       63 |         0 |     3 |      3 |  17 | 22e1b5990a8fdeb4 |    1 |
| 52 | 0xd000 |       63 |         0 |     0 |      0 |   0 | 92cb1085e4bccf85 |    0 |

Done
> ipaddr
fd21:8892:8f38:9713:0:ff:fe00:fc00
fd21:8892:8f38:9713:0:ff:fe00:d000
fd21:8892:8f38:9713:730e:e603:5753:69b
fe80:0:0:0:90cb:1085:e4bc:cf85
Done
> ping fd21:8892:8f38:9713:0:ff:fe00:2800
16 bytes from fd21:8892:8f38:9713:0:ff:fe00:2800: icmp_seq=1 hlim=64 time=3ms
1 packets transmitted, 1 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 3/3
.0/3 ms.
Done
>
```

```
0 packets dropped by kernel
root@56872738332e:/# tcpdump -i any -n udp portrange 9001-9002
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length
 262144 bytes
20:11:41.485585 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, leng
th 71
20:11:43.628338 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, leng
th 71
20:11:45.675125 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, leng
th 41
20:11:45.675200 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, leng
th 6
20:11:45.676768 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, leng
th 41
20:11:45.676813 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, leng
th 6
```

2: root@56872738332e: /

```
router
Done
> ipaddr
fd21:8892:8f38:9713:0:ff:fe00:2800
fd21:8892:8f38:9713:83a8:abd4:7439:9db1
fe80:0:0:0:20e1:b599:a8f:deb4
Done
> ping fd21:8892:8f38:9713:0:ff:fe00:d000
16 bytes from fd21:8892:8f38:9713:0:ff:fe00:d000: icmp_seq=1 hlim=64 time=2ms
1 packets transmitted, 1 packets received. Packet loss = 0.0%. Round-trip min/avg/max = 2/2.0/2 ms.
Done
>
```

Simulate a Thread network: Test the network

Now that you can successfully ping between two emulated Thread devices, test the mesh network by taking one node offline.

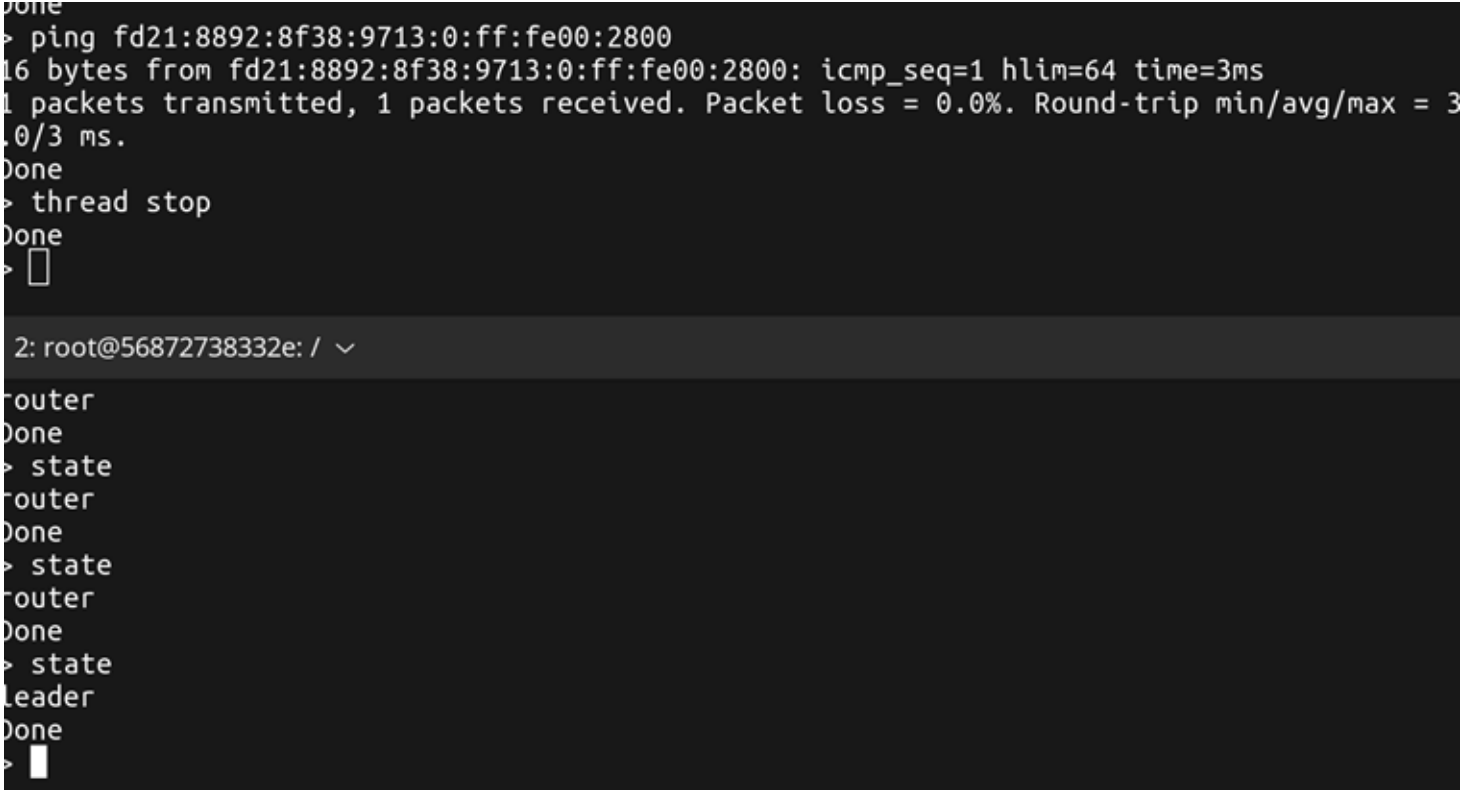
Return to Node 1 and stop Thread:

```
> thread stop
        Done
```

Switch to Node 2 and check the state. Within two minutes, Node 2 detects that the leader (Node 1) is offline, and you should see Node 2 transition to be the `leader` of the network:

```
> state
        router
        Done
        ...
> state
        leader
        Done
```

Simulate a Thread network: Test the network



Simulate a Thread network: Test the network

Once confirmed, stop Thread and factory reset Node 2 before exiting back to the Docker `bash` prompt. A factory reset is done to ensure that the Thread network credentials we used in this exercise are not carried over to the next exercise.

```
> thread stop
        Done
> factoryreset
>
> exit
root@c0f3912a74ff:/#
```

You may have to press `enter` a few times to bring the `>` prompt back after a `factoryreset` command. Do not exit the Docker container.

Also factory reset and exit Node 1:

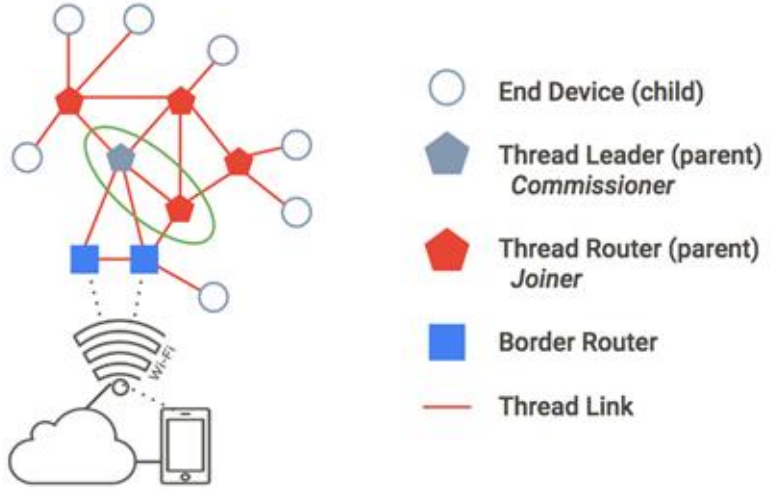
```
> factoryreset
>
> exit
 root@c0f3912a74ff:/#
```

Authenticate nodes with Commissioning

To route global IPv6 traffic between them (and the Internet via a Thread border router), nodes must be authenticated.

In order to authenticate, one device must act as a Commissioner. The Commissioner is the currently elected authentication server for new Thread devices, and the authorizer for providing the network credentials required for the devices to join the network.

In this example, we will use the same two-node topology as before. For authentication, the Thread Leader will act as the Commissioner, the Thread Router as a Joiner.



Authenticate nodes with Commissioning

For each Node (terminal window) in the remaining exercises, make sure you are running the Docker container with the OpenThread build. If continuing from the previous exercise, you should still have two `bash` prompts within the same Docker container already open. If not, see the Docker Troubleshooting step, or simply redo the Simulate a Thread network exercise.

1. Create a network

In Node 1, spawn the CLI process:

```
root@c0f3912a74ff:/# /openthread/build/examples/apps/cli/ot-cli-ftd 1
```

Note: If you don't see the `>` prompt after running this command, press `enter`.

Create a new Operational Dataset, commit it as the active one, and start Thread:

```
> dataset init new
Done
> dataset
Active Timestamp: 1
Channel: 12
Channel Mask: 07fff800
Ext PAN ID: e68d05794bf13052
Mesh Local Prefix: fd7d:ddf7:877b:8756/64
Network Key: a77fe1d03b0e8028a4e13213de38080e
Network Name: OpenThread-8f37
PAN ID: 0x8f37
PSKc: f9debbc1532487984b17f92cd55b21fc
Security Policy: 0, onrcb
Done
```

Authenticate nodes with Commissioning

Commit this dataset as the active one:

```
> dataset commit active
Done
```

Bring up the IPv6 interface:

```
> ifconfig up
Done
```

Start Thread protocol operation:

```
> thread start
Done
```

Wait a few seconds and verify that the device has become a Thread Leader:

```
> state
leader
Done
```

Authenticate nodes with Commissioning: Start the Commissioner role

While still on Node 1, start the Commissioner role:

```
> commissioner start
        Done
```

Allow any Joiner (by using the `*` wildcard) with the `J01NME` Joiner Credential to commission onto the network. A Joiner is a device that is added by a human administrator to a commissioned Thread Network.

```
> commissioner joiner add * J01NME
        Done
```

Authenticate nodes with Commissioning: Start the Joiner role

In a second terminal window, in the Docker container, spawn a new CLI process. This is Node 2.

```
root@c0f3912a74ff:/# /openthread/build/examples/apps/cli/ot-cli-ftd 2
```

On Node 2, enable the Joiner role using the `J01NME` Joiner Credential.

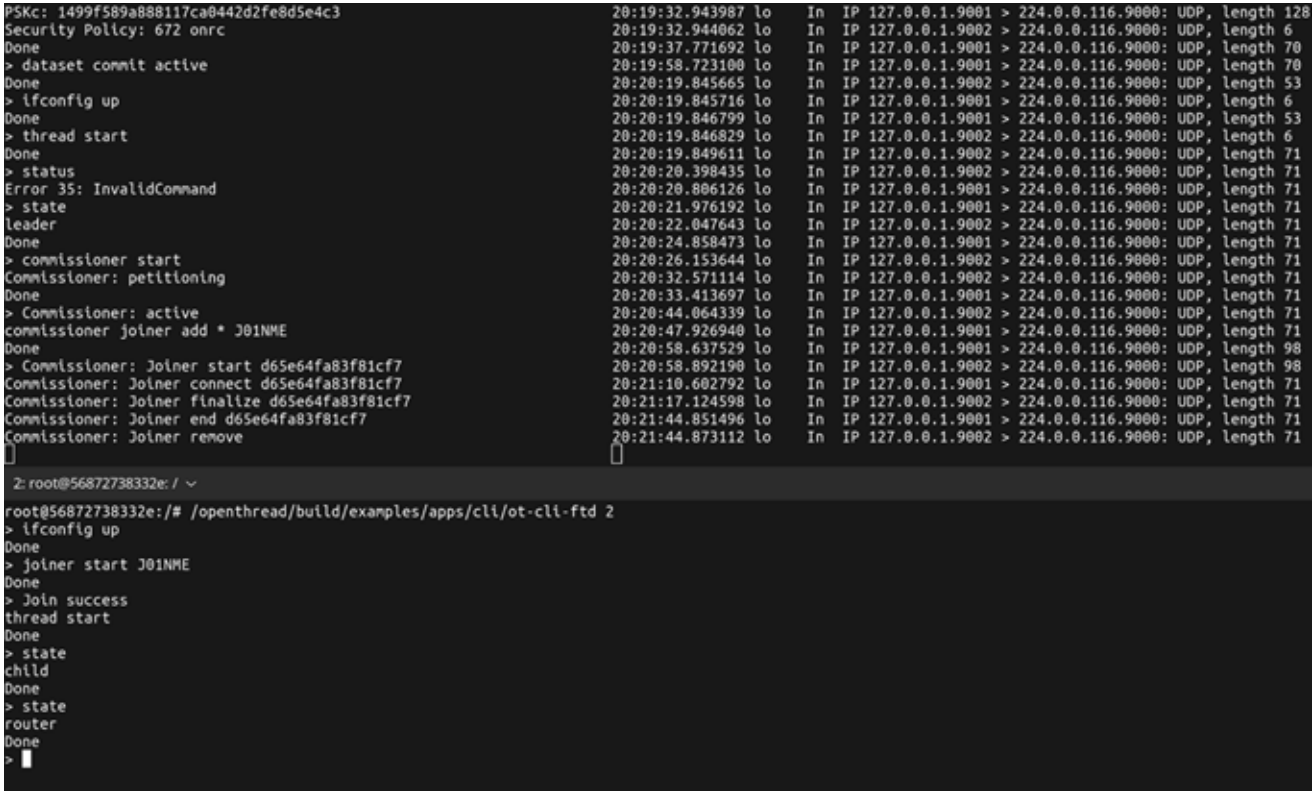
```
> ifconfig up
Done
> joiner start J01NME
Done
```

```
... wait a few seconds for confirmation ...
Join success
```

As a Joiner, the device (Node 2) has successfully authenticated itself with the Commissioner (Node 1) and received the Thread Network credentials.
Now that Node 2 is authenticated, start Thread:

```
> thread start
Done
```

Authenticate nodes with Commissioning: Start the Joiner role



Authenticate nodes with Commissioning: Reset configuration

To prepare for the next exercise, reset the configuration. On each Node, stop Thread, do a factory reset, and exit the emulated Thread device:

```
> thread stop
        Done
> factoryreset
>
> exit
root@c0f3912a74ff:/#
```

You may have to press `enter` a few times to bring the `>` prompt back after a `factoryreset` command.

Manage the network with OpenThread Daemon

For this exercise, we are going to simulate one CLI instance (a single embedded SoC Thread device) and one Radio Co-Processor (RCP) instance.

`ot-daemon` is a mode of the OpenThread Posix app that uses a UNIX socket as input and output, so that OpenThread core can run as a service. A client can communicate with this service by connecting to the socket using the OpenThread CLI as the protocol.

`ot-ctl` is a CLI provided by `ot-daemon` to manage and configure the RCP. Using this, we'll connect the RCP to the network created by the Thread device.

For each Node (terminal window) in this exercise, make sure you are running the Docker container with the OpenThread build. If continuing from the previous exercise, you should have two `bash` prompts within the same Docker container already open. If not, see the Docker Troubleshooting step.

Use ot-daemon

This exercise will use three terminal windows, corresponding to the following:

1. CLI instance of simulated Thread device (Node 1)
2. `ot-daemon` process
3. `ot-ctl` CLI instance

Manage the network with OpenThread Daemon: Start Node 1

Start Node 1

In the first terminal window, spawn the CLI process for your emulated Thread device:

```
root@c0f3912a74ff:/# /openthread/build/examples/apps/cli/ot-cli-ftd 1
```

Create a new Operational Dataset, commit it as the active one, and start Thread:

```
> dataset init new
Done

> dataset
Active Timestamp: 1
Channel: 13
Channel Mask: 07fff800
Ext PAN ID: 97d584bcd493b824
Mesh Local Prefix: fd55:cf34:dea5:7994/64
Network Key: ba6e886c7af50598df1115fa07658a83
Network Name: OpenThread-34e4
PAN ID: 0x34e4
PSKc: 38d6fd32c866927a4dfcc06d79ae1192
Security Policy: 0, onrcb
Done
```

Manage the network with OpenThread Daemon: Start Node 1

Commit this dataset as the active one:

```
> dataset commit active
                    Done
```

Bring up the IPv6 interface:

```
> ifconfig up
                    Done
```

Start Thread protocol operation:

```
> thread start
                    Done
```

View the IPv6 addresses assigned to Node 1's Thread interface:

```
> ipaddr
                    fd55:cf34:dea5:7994:0:ff:fe00:fc00
                    fd55:cf34:dea5:7994:0:ff:fe00:d000
                    fd55:cf34:dea5:7994:460:872c:e807:c4ab
                    fe80:0:0:0:9cd8:aab6:482f:4cdc
                    Done
```

Manage the network with OpenThread Daemon: Start ot-daemon

In the second terminal window, create a `tun` device node and set read/write permissions:

```
root@c0f3912a74ff:/# mkdir -p /dev/net && mknod /dev/net/tun c 10 200
root@c0f3912a74ff:/# chmod 600 /dev/net/tun
```

This device is used for packet transmission and receipt in virtual devices. You may get an error if the device has already been created—this is normal and can be ignored.

Start `ot-daemon` for an RCP node, which we'll call Node 2. Use the `-v` verbose flag so you can see log output and confirm that it is running:

```
root@c0f3912a74ff:/# /openthread/build/posix/src/posix/ot-daemon -v \
'spinel+hdlc+forkpty:///openthread/build/examples/apps/ncp/ot-rcp?forkpty-arg=2'
```

When successful, `ot-daemon` in verbose mode generates output similar to the following:

```
ot-daemon[31]: Running OPENTHREAD/297a880; POSIX; Feb  1 2022 04:43:39
ot-daemon[31]: Thread version: 3
ot-daemon[31]: Thread interface: wpan0
ot-daemon[31]: RCP version: OPENTHREAD/297a880; SIMULATION; Feb  1 2022 04:42:50
```

Manage the network with OpenThread Daemon: Use ot-ctl to join the network

We haven't commissioned Node 2 (the `ot-daemon` RCP) to any Thread network yet. This is where `ot-ctl` comes in. `ot-ctl` uses the same CLI as the OpenThread CLI app. Therefore, you can control `ot-daemon` nodes in the same manner as the other simulated Thread devices.

Open a third terminal window and execute the existing container:

```
$ docker exec -it codelab_otsim_ctnr bash
```

Once in the container, start `ot-ctl`:

```
root@c0f3912a74ff:/# /openthread/build/posix/src/posix/ot-ctl
```

You'll use `ot-ctl` in this third terminal window to manage Node 2 (the RCP node) that you started in the second terminal window with `ot-daemon`. Check the `state` of Node 2:

```
> state
                disabled
                Done
```

Manage the network with OpenThread Daemon: Use ot-ctl to join the network

Get Node 2's `eui64`, to restrict joining to the specific Joiner:

```
> eui64
                18b4300000000001
                Done
```

On Node 1 (first terminal window), start the Commissioner and restrict joining to only that eui64:

```
> commissioner start
                Done

> commissioner joiner add 18b4300000000001 J01NME
                Done
```

In the third terminal window, bring up the network interface for Node 2 and join the network:

```
> ifconfig up
                Done

> joiner start J01NME
                Done
... wait a few seconds for confirmation ...
                Join success
```

Manage the network with OpenThread Daemon: Use ot-ctl to join the network

As a Joiner, the RCP (Node 2) has successfully authenticated itself with the Commissioner (Node 1) and received the Thread Network credentials.

Now join Node 2 to the Thread network (again, in the third terminal window):

```
> thread start
                    Done
```

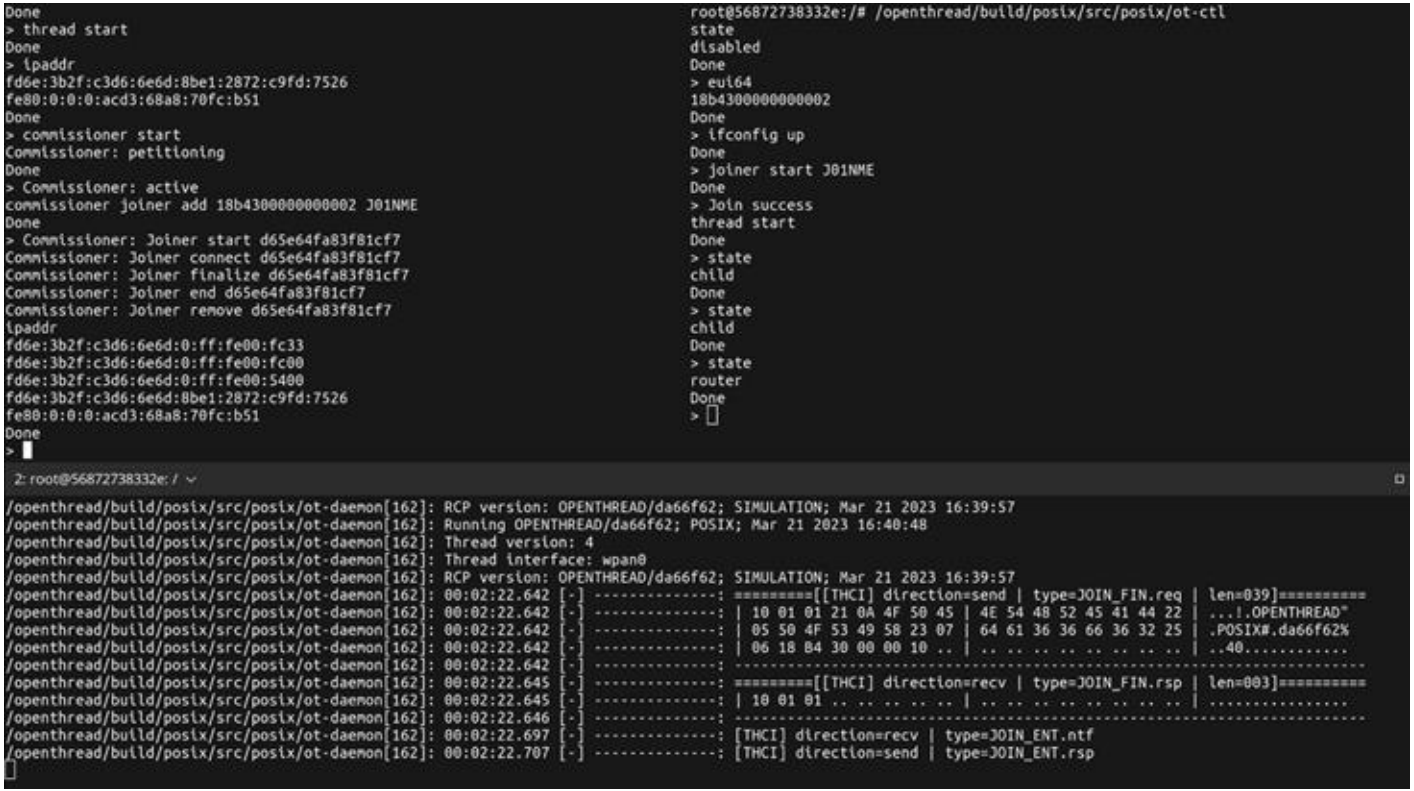
Validate network authentication

In the third terminal, check the `state` on Node 2, to validate that it has now joined the network. Within two minutes, Node 2 transitions from `child` to `router`:

```
> state
                    child
                    Done

...
> state
                    router
                    Done
```

Manage the network with OpenThread Daemon: Use ot-ctl to join the network



Manage the network with OpenThread Daemon: Use ot-ctl to join the network

Validate connectivity

In the third terminal window, quit `ot-ctl` by using either Ctrl+D or `exit` command, and return to the container's `bash` console.

From this console, ping Node 1, using its EID with the `ping6` command. If the `ot-daemon` RCP instance is successfully joined to and communicating with the Thread network, the ping succeeds:

```
root@c0f3912a74ff:/# ping6 -c 4 fd55:cf34:dea5:7994:460:872c:e807:c4ab
PING fd55:cf34:dea5:7994:460:872c:e807:c4ab (fd55:cf34:dea5:7994:460:872c:e807:c4ab): 56 data bytes
64 bytes from fd55:cf34:dea5:7994:460:872c:e807:c4ab: icmp_seq=0 ttl=64 time=4.568 ms
64 bytes from fd55:cf34:dea5:7994:460:872c:e807:c4ab: icmp_seq=1 ttl=64 time=6.396 ms
64 bytes from fd55:cf34:dea5:7994:460:872c:e807:c4ab: icmp_seq=2 ttl=64 time=7.594 ms
64 bytes from fd55:cf34:dea5:7994:460:872c:e807:c4ab: icmp_seq=3 ttl=64 time=5.461 ms
--- fd55:cf34:dea5:7994:460:872c:e807:c4ab ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 4.568/6.005/7.594/1.122 ms
```

Check Ping with tcpdump

To view ping packets being sent between the two emulated Thread nodes using tcpdump, you will need to start tcpdump inside the Docker container and monitor traffic on the UDP port associated with the IEEE 802.15.4 radio emulation. Since you used file descriptors 1 and 2 to start the emulated nodes, the corresponding UDP ports will be 9001 and 9002.

Open a new terminal and connect to the existing Docker container:

Start tcpdump to monitor UDP traffic on ports 9001 and 9002:

```
root@c0f3912a74ff:/# tcpdump -i any -n udp portrange 9001-9002
```

```
19:16:15.305474 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, length 71
19:16:22.366795 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, length 79
19:16:43.530645 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, length 79
19:16:47.319398 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, length 71
19:16:49.626294 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, length 71
19:17:04.565829 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, length 79
19:17:07.715436 lo    In  IP 127.0.0.1.9001 > 224.0.0.116.9000: UDP, length 71
19:17:16.405568 lo    In  IP 127.0.0.1.9002 > 224.0.0.116.9000: UDP, length 71
```